

CAHIER DES CHARGES

Projet de Fin d'Etudes

Systeme de Detection d'Intrusion Reseau

Base sur l'Intelligence Artificielle avec Tableau de Bord Web



Edit with WPS Office

1. CONTEXTE ET PRESENTATION DU PROJET

1.1 Contexte General

La cybersécurité est devenue un enjeu stratégique majeur pour toutes les organisations. Avec l'explosion du trafic réseau, la multiplication des cybermenaces (DDoS, ransomware, exfiltration de données) et la sophistication croissante des attaquants, les solutions de détection traditionnelles basées sur des signatures statiques ne suffisent plus.

Ce projet de fin d'études propose la conception et le développement d'un Système de Détection d'Intrusion Réseau (NIDS) nouvelle génération, exploitant les capacités de l'Intelligence Artificielle et du Machine Learning pour détecter en temps réel les comportements anormaux et les attaques sur le réseau.

1.2 Problematique

Les systèmes de sécurité traditionnels présentent des limitations majeures :

- Détection uniquement par signatures : incapacité à identifier les attaques zero-day
- Nombre élevé de faux positifs générant une fatigue opérationnelle
- Absence de visibilité centralisée et en temps réel sur l'état du réseau
- Manque d'outils accessibles intégrant détection réseau et analyse d'applications web

Comment concevoir un système intelligent, performant et accessible qui détecte les intrusions réseau en temps réel, réduit les faux positifs et offre une interface de supervision claire pour les équipes de sécurité ?

1.3 Objectifs du Projet

Les objectifs principaux sont :

1. Capturer et analyser le trafic réseau en temps réel avec extraction automatique de caractéristiques
2. Entraîner et déployer un modèle de Machine Learning pour la classification des attaques
3. Développer un tableau de bord web interactif pour la supervision et la gestion des alertes
4. Intégrer un module de scan de vulnérabilités pour les actifs réseau
5. Containeriser la solution avec Docker pour faciliter le déploiement

2. PERIMETRE ET PORTEE DU PROJET

2.1 Inclus dans le Projet

- Moteur de capture de paquets reseau (live + fichiers PCAP)
- Pipeline d'extraction et de normalisation de caracteristiques
- Moteur de detection base sur le Machine Learning (Random Forest + XGBoost)
- Systeme d'alertes en temps reel avec niveaux de severite
- API REST backend (FastAPI)
- Tableau de bord web React avec visualisations en temps reel
- Module de scan de ports et de vulnerabilites
- Base de donnees PostgreSQL + Redis
- Containerisation Docker avec Docker Compose
- Documentation technique et rapport de projet

2.2 Hors Perimetre

- Systeme de prevention (IPS) – uniquement detection (IDS)
- Integration avec des SIEM commerciaux (Splunk, QRadar)
- Application mobile
- Support des protocoles industriels (SCADA, Modbus)

3. SPECIFICATIONS FONCTIONNELLES

3.1 Module de Capture Reseau

Le systeme doit etre capable de capturer et analyser le trafic reseau avec les fonctionnalites suivantes :

ID	Fonctionnalite	Priorite	Description
F-01	Capture en temps reel	Haute	Capture des paquets sur interfaces reseau via Scapy/libpcap
F-02	Import PCAP	Haute	Analyse de fichiers de capture existants (.pcap, .pcapng)
F-03	Filtrage de protocoles	Moyenne	Support TCP, UDP, ICMP, HTTP, DNS, HTTPS
F-04	Extraction de caracteristiques	Haute	44 features extraites par flux (duree, taille, flags, frequence)
F-05	Multi-interface	Basse	Ecoute simultanee sur plusieurs interfaces reseau

3.2 Moteur de Detection IA

ID	Fonctionnalite	Priorite	Description
----	----------------	----------	-------------

ID	Fonctionnalite	Priorite	Description
F-06	Classification binaire	Haute	Detection normal vs. attaque en temps reel
F-07	Classification multi-classes	Haute	Identification du type : DoS, Scan, Brute Force, Exfiltration
F-08	Score de confiance	Moyenne	Probabilite associee a chaque prediction
F-09	Modele ensemble	Haute	Combinaison Random Forest + XGBoost pour robustesse
F-10	Mise a jour du modele	Basse	Re-entrainement planifie sans interruption de service

3.3 Systeme d'Alertes

ID	Fonctionnalite	Priorite	Description
F-11	Generation d'alertes	Haute	Creation automatique lors de detection avec metadonnees completes
F-12	Niveaux de severite	Haute	4 niveaux : Info, Faible, Moyen, Critique
F-13	Notification temps reel	Haute	Push via WebSocket vers le tableau de bord
F-14	Notification email	Moyenne	Envoi de mails pour alertes critiques
F-15	Acquittement	Moyenne	Possibilite de marquer les alertes comme traitees
F-16	Export	Basse	Export CSV/PDF des rapports d'alertes

3.4 Tableau de Bord Web

ID	Fonctionnalite	Priorite	Description
F-17	Vue d'ensemble	Haute	KPIs en temps reel : paquets/s, alertes actives, taux detection
F-18	Graphique de trafic	Haute	Courbe temporelle du volume de trafic par protocole
F-19	Carte GeoIP	Moyenne	Cartographie mondiale des sources d'attaques
F-20	Feed d'alertes	Haute	Liste filtreable et triable des alertes en temps reel
F-21	Detail d'alerte	Haute	Vue detaillee avec IP source/dest, protocole, payload
F-	Statistiques ML	Moyenne	Metriques du modele : precision, recall, matrice de

ID	Fonctionnalite	Priorite	Description
22			confusion
F-23	Gestion utilisateurs	Haute	Authentification JWT, roles Admin/Analyste/Lecteur
F-24	Scanner de ports	Moyenne	Interface pour lancer des scans Nmap sur des cibles

4. SPECIFICATIONS NON FONCTIONNELLES

4.1 Performance

Critere	Exigence	Methode de Verification
Throughput de capture	≥ 1 Gbps sans perte de paquets	Test avec iperf3 en lab
Latence de detection	< 500 ms du paquet a l'alerte	Mesure end-to-end avec timestamps
Precision du modele	$\geq 97\%$ sur dataset CICIDS2017	Evaluation sur jeu de test isole
Rappel (Recall)	$\geq 95\%$ pour attaques critiques	Matrice de confusion sur test set
Faux positifs	$< 3\%$ sur trafic benin	Test sur captures de production
Temps de chargement UI	< 2 secondes (dashboard)	Test avec Lighthouse / Chrome DevTools

4.2 Securite

- Authentification JWT avec expiration et refresh tokens
- Chiffrement TLS 1.3 pour toutes les communications API
- Hachage des mots de passe avec bcrypt (cost factor ≥ 12)
- Controle d'accès basé sur les rôles (RBAC)
- Journalisation de toutes les actions utilisateurs (audit log)
- Protection contre les injections SQL (ORM SQLAlchemy)
- Rate limiting sur les endpoints API

4.3 Disponibilite et Fiabilite

- Disponibilite cible : 99.5% (hors maintenance planifiee)
- Persistence des alertes en base de donnees meme en cas de coupure reseau
- Reconnexion automatique du WebSocket en cas de deconnexion
- Healthcheck Docker pour redemarrage automatique des services

4.4 Maintainabilité

- Architecture microservices avec separation claire des responsabilites
- Code documente (docstrings Python, JSDoc React)
- Tests unitaires (couverture $\geq 80\%$) et tests d'integration
- Logs structurees en JSON avec niveaux (DEBUG/INFO/WARNING/ERROR)
- Configuration externalisee via fichiers .env

5. ARCHITECTURE TECHNIQUE

5.1 Vue d'ensemble de l'Architecture

Le systeme suit une architecture en couches avec separation nette entre la collecte de donnees, le traitement IA, le stockage et la presentation :

Couche	Composant	Technologie	Role
Collecte	Packet Engine	Python / Scapy	Capture et pre-traitement des paquets
Traitement	Feature Extractor	Python / Pandas	Extraction des 44 caracteristiques par flux
IA	ML Engine	scikit-learn XGBoost	/ Classification et detection des attaques
Stockage	Database	PostgreSQL Redis	+ Persistance des alertes et cache temps reel
API	REST Backend	Python / FastAPI	Exposition des endpoints et logique metier
Temps reel	WebSocket	FastAPI WebSocket	/ Push des alertes vers le frontend
Presentation	Dashboard	React / Tailwind CSS	Interface utilisateur et visualisations
Infra	Containerisation	Docker / Compose	Deploiement et orchestration des services

5.2 Stack Technologique

Categorie	Technologie	Version	Justification
Langage backend	Python	3.11+	Ecosysteme ML riche, maturite pour la securite reseau
Framework API	FastAPI	0.110+	Performances asynchrones, documentation auto Swagger
Capture reseau	Scapy	2.5+	Flexibilite et support de tous les protocoles
Machine	scikit-learn	1.4+	Modeles robustes, pipeline ML standardise

Categorie	Technologie	Version	Justification
Learning			
Boosting	XGBoost	2.0+	Performance superieure sur donnees tabulaires
Base de donnees	PostgreSQL	16+	Fiabilite, performance, support JSONB
Cache	Redis	7+	Pub/Sub pour WebSocket, cache haute performance
Frontend	React	18+	Composants reutilisables, ecosysteme riche
UI Framework	Tailwind CSS	3+	Personnalisation rapide, design system coherent
Graphiques	Chart.js Recharts	/ 4+	Visualisations temps reel performantes
Containerisation	Docker Compose	+ 25+	Portabilite et reproductibilite du deploiement

5.3 Dataset d'Entrainement

Le modele sera entraine et evalue sur le dataset CICIDS2017 (Canadian Institute for Cybersecurity) :

Caracteristique	Detail
Nom	CICIDS2017 – CIC Intrusion Detection Evaluation Dataset
Source	University of New Brunswick (UNB) – unb.ca/cic/datasets
Taille	~2.8 GB – 2.8 millions d'enregistrements
Classes d'attaques	DoS/DDoS, PortScan, Brute Force, Web Attacks, Infiltration, Botnet
Features	78 caracteristiques reseau extraites avec CICFlowMeter
Repartition	~80% train / 20% test avec stratification par classe
Benchmark	Precision attendue $\geq 97\%$ (litterature scientifique)

6. ORGANISATION ET PLANIFICATION

6.1 Equipe Projet

Role	Responsabilites	Competences Requises
Etudiant (Developpeur Principal)	Conception, developpement, tests, documentation et soutenance	Python, ML, React, Docker, Securite reseau
Encadrant Academique	Validation des specifications, suivi	Cybersecurite, Machine

Role	Responsabilites	Competences Requises
	methodologique, evaluation	Learning, logicielle Architecture
Co-encadrant applicable)	(si Expertise technique specifique, validation des choix IA	ML/Deep Securite Learning, offensive et defensive

6.2 Planning Previsionnel (8 Semaines)

Phase	Semaine	Livrables	Criteres de Validation
Phase 1 : Setup & Capture	S1 – S2	Moteur de capture, pipeline d'extraction, notebook EDA	Capture fonctionnelle, extraction de 44 features validee
Phase 2 : Modele ML	S3 – S4	Modele entraine, rapport d'evaluation, API prediction	Precision $\geq 97\%$, F1-score ≥ 0.96 sur test set
Phase 3 : Backend API	S5	API REST complete, base de donnees, systeme d'alertes	Tous les endpoints documentes, tests unitaires OK
Phase 4 : Frontend	S6 – S7	Dashboard React complet, WebSocket, cartes GeolP	Alertes affichees < 500ms, interface responsive
Phase 5 : Integration	S8	Systeme integre, Docker Compose, tests end-to-end, docs	Deploiement en 1 commande, rapport technique final

6.3 Jalons Cles

- J1 – Fin S2 : Premier flux reseau detecte et classe par le modele
- J2 – Fin S4 : Rapport d'evaluation ML valide par l'encadrant
- J3 – Fin S6 : Demo du tableau de bord avec alertes en temps reel
- J4 – Fin S8 : Livraison complete + soutenance de stage

7. LIVRABLES DU PROJET

7.1 Livrables Techniques

#	Livrable	Format	Echeance
L-01	Code source complet avec historique Git	GitHub Repository	Semaine 8
L-	Modele ML entraine et serialise	Fichier .pkl / .joblib	Semaine 4

#	Livrable	Format	Echeance
02			
L-03	Configuration Docker Compose	docker-compose.yml	Semaine 8
L-04	Documentation (Swagger/OpenAPI) API	URL accessible	Semaine 5
L-05	Suite de tests (unit + integration)	pytest + Jest	Semaine 8
L-06	Guide d'installation et de déploiement	README.md	Semaine 8

7.2 Livrables Documentaires

#	Document	Contenu	Echeance
D-01	Cahier des Charges (ce document)	Specifications complètes du projet	Semaine 1
D-02	Rapport de conception	Architecture détaillée, choix techniques justifiés	Semaine 4
D-03	Rapport d'évaluation ML	Métriques, comparaisons, analyse des erreurs	Semaine 4
D-04	Rapport final de PFE	Document complet (80–120 pages)	Semaine 8
D-05	Présentation de soutenance	Diaporama 20–25 slides	Soutenance

8. CONTRAINTES ET RISQUES

8.1 Contraintes Techniques

- Le système doit fonctionner sur Linux (Ubuntu 22.04 LTS minimum)
- Mémoire RAM minimum : 8 Go pour l'exécution complète en Docker
- La capture de paquets nécessite des privilèges root/sudo sur la machine hôte
- Le dataset CICIDS2017 requiert environ 15 Go d'espace disque pour le traitement
- Les tests de pénétration doivent être réalisés uniquement sur des environnements isolés et autorisés

8.2 Contraintes Ethiques et Légales

IMPORTANT : Ce projet doit respecter strictement les contraintes suivantes :

- Tous les tests de génération de trafic malveillant doivent être effectués sur un

reseau isole (lab virtuel)

- L'utilisation d'outils de scan (Nmap, Scapy) est interdite sur des reseaux tiers sans autorisation explicite
- Les donnees capturees ne doivent pas contenir d'informations personnelles en dehors du lab
- Le projet s'inscrit dans un cadre purement educatif et defensif

8.3 Analyse des Risques

Risque	Probabilite	Impact	Mitigation
Taux de faux positifs trop eleve	Moyenne	Haute	Tuning des hyperparametres, seuil de decision ajustable
Performance insuffisante sur trafic reel	Faible	Haute	Test progressif, feature engineering ameliore
Depassement du planning	Moyenne	Moyenne	Planning avec marge, priorisation des features MVP
Problemes de privileges pour capture	Faible	Moyenne	Utilisation de machines virtuelles dediees
Dataset non representatif	Faible	Haute	Completer avec UNSW-NB15 ou CIC-DDoS2019

9. CRITERES D'EVALUATION ET DE SUCCES

9.1 Criteres Techniques (60%)

Critere	Poids	Indicateur de Succes
Fonctionnalite et completeude du systeme	20%	Toutes les fonctionnalites F-01 a F-24 implementees et operationnelles
Performance du modele ML	20%	Precision $\geq 97\%$, Recall $\geq 95\%$, F1 ≥ 0.96 sur CICIDS2017
Qualite du code et architecture	10%	Code propre, documente, tests $\geq 80\%$ de couverture
Securite de l'application web	10%	Authentification, chiffrement, protection OWASP Top 10

9.2 Criteres Academiques (40%)

Critere	Poids	Indicateur de Succes
Qualite du rapport de PFE	20%	Redaction claire, etat de l'art solide, analyse critique

Critere	Poids	Indicateur de Succes
Presentation et soutenance	10%	Maitrise du sujet, clarte de l'expose, reponses aux questions
Innovation et valeur ajoutee	10%	Apport original par rapport a l'existant, perspectives d'evolution

10. REFERENCES ET RESSOURCES

10.1 Datasets

- CICIDS2017 : <https://www.unb.ca/cic/datasets/ids-2017.html>
- NSL-KDD : <https://www.kaggle.com/datasets/hassan06/nsldata>
- UNSW-NB15 : <https://research.unsw.edu.au/projects/unswnb15-dataset>

10.2 Articles et Publications de Reference

- Yin, C. et al. (2017). A Deep Learning Approach for Intrusion Detection. IEEE Access
- Sommer, R. & Paxson, V. (2010). Outside the Closed World: On Using ML for Network IDS. IEEE S&P
- Ring, M. et al. (2019). A Survey of Network-based Intrusion Detection Data Sets. Computers & Security

10.3 Outils et Librairies

- Scapy (capture reseau) : <https://scapy.net>
- FastAPI (backend) : <https://fastapi.tiangolo.com>
- scikit-learn (ML) : <https://scikit-learn.org>
- XGBoost : <https://xgboost.readthedocs.io>
- React : <https://react.dev>
- Docker : <https://docs.docker.com>

11. VALIDATION ET SIGNATURES

Ce cahier des charges engage les parties sur les specifications du projet. Toute modification majeure devra faire l'objet d'un avenant signe par les deux parties.

Etudiant	Encadrant Academique
Nom & Prenom : _____	Nom & Prenom : _____

Date : _____	Grade / Titre : _____
Signature :	Date : _____
	Signature & Cachet :

Document prepare pour usage academique – Projet de Fin d'Etudes 2025–2026

